강의 자료 · 쉬운 요약

Claude Code 팀 에이전트, 진짜 쉽게

클로드 여러 명을 한 팀으로 · 단계별 용어 풀이와 프롬프트

Lead Teammates Task list



목차

들어가며	이 자료 읽는 법	3
시작하기	팀 에이전트가 뭔데?	4
용어	어려운 용어 → 쉬운 말 번역표	7
한눈에 보기	혼자 · 심부름꾼 · 팀	9
1단계	심부름꾼 복습 (Subagents)	11
2단계	언제 팀이 필요해? (Subagents vs Agent Teams)	14
3단계	팀 기능 켜기	17
4단계	첫 팀 만들기 (Spawn · 팀원 현황판)	20
5단계	화면 모드 (한 화면 · 분할 화면)	22
6단계	팀 구성 정하기 (인원 · 이름 · 모델)	25
7단계	업무 지시서 잘 쓰기 (Spawn prompt)	27
8단계	공유 할 일 보드 (Task list)	29
9단계	팀원끼리 대화하기 (메신저 · 토론)	31
10단계	설계 먼저, 권한은 내가 (Plan mode · Permissions)	33
11단계	역할 카드 재사용 (Subagent definitions → 팀원)	35
12단계	퇴근·완료 검문소 (Hooks)	38
13단계	파일 충돌 피하기 (파일 나누기 · Worktree)	41
14단계	비용 · 운영 · 한계	44
부록 A	상황별 프롬프트 모음	47
부록 B	설정 · 키 치트시트	49
부록 C	팀 운영 10계명	51

들어가며

이 자료 읽는 법

클로드 **여러 명을 한 팀으로** 일하게 만드는 Claude Code 의 **Agent Teams(팀 에이전트)** 입문서입니다. ⚠️ 팀 에이전트는 아직 **실험 기능(experimental)** 입니다. 화면·이름이 바뀔 수 있으니, 수업 전 공식 문서(code.claude.com/docs/en/agent-teams)를 한 번 확인하세요. 각 단계는 같은 순서로 되어 있습니다.

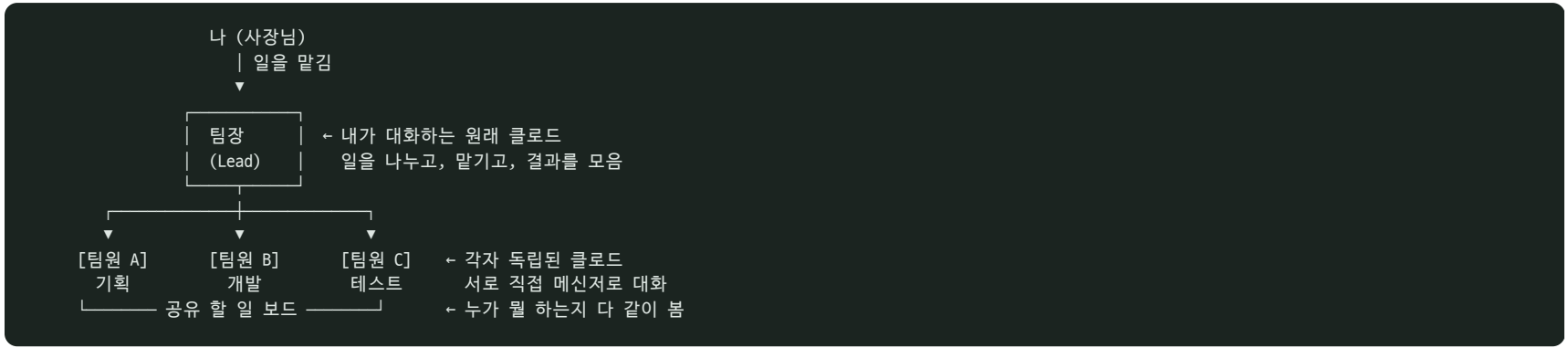
1. 📖 **이게 뭐야?** — 단계에 나오는 용어를 하나씩 풀어서 설명
2. ❓ **왜 필요해?** — 안 하면 무슨 일이 생기는지
3. 🛠️ **직접 해보기** — 실제로 치는 설정과 프롬프트
4. ⚠️ **자주 하는 실수**
5. 💬 **클로드에게 이렇게 말하기** — 복붙용 프롬프트

팀 에이전트가 뭔데?

📖 이게 뭐야?

지금까지 우리는 **클로드 한 명**에게 일을 시켰습니다. 똑똑하지만, 한 번에 한 가지 생각만 합니다.

팀 에이전트 = 클로드 여러 명으로 꾸린 회사 한 팀입니다.



구성	쉬운 말	하는 일
Team lead	팀장	내가 대화하는 원래 클로드. 팀원을 뽑고, 일을 나누고, 결과를 합침
Teammates	팀원	각자 **독립된 클로드**. 자기 일을 하고 서로 대화함
Task list	공유 할 일 보드	해야 할 일 목록. 팀원이 골라서 맡고 완료 표시
Mailbox	사내 메신저	팀원끼리, 팀장과 팀원이 주고받는 메시지

❓ 왜 필요해?

클로드 한 명은 **첫 번째로 떠오른 답에 꽂히는 버릇**이 있습니다. 버그 원인을 찾다가 그럴듯한 하나를 찾으면 거기서 멈춥니다. 코드 리뷰를 시키면 보안이든 성능이든 한 가지 쪽만 깊게 봅니다.

팀으로 일하면:

- **동시에 여러 각도**로 봅니다 (보안 담당, 성능 담당, 테스트 담당)
- **서로 반박**합니다 ("네 가설은 이 로그랑 안 맞아")
- 각자 자기 **머릿속(컨텍스트)이 따로**라서, 한 명이 다 기억하느라 헷갈리지 않습니다

⚠️ 먼저 알아둘 것 3가지

	내용
실험 기능	기본으로 꺼져 있음. 설정으로 켜야 함 (3단계)
토큰을 많이 씀	팀원 한 명 = 클로드 한 명. 3명이면 대략 3배 이상. 공식 문서 기준 팀원이 계획 모드로 돌면 **약 7배**
항상 정답은 아님	순서대로 해야 하는 일, 같은 파일을 고치는 일은 **혼자 하는 게 더 빠름**

팀 에이전트 = 회의가 필요한 큰일에만 꾸리는 프로젝트 팀. 매일의 잔일에 팀을 꾸리면 회의비(토큰)만 나갑니다.

용어

어려운 용어 → 쉬운 말 번역표

원래 용어	쉬운 말	자세한 설명
Agent Teams	**클로드 팀**	시작하기
Team lead	**팀장**	시작하기
Teammate	**팀원**	시작하기
Subagent	**심부름꾼** (결과만 가져옴)	1~2단계
Context window	**각자의 머릿속**	2단계
Spawn	**팀원 뽑기 / 투입**	4단계
Agent panel	**팀원 현황판**	4단계
In-process / Split panes	**한 화면 / 분할 화면**	5단계
Spawn prompt	**업무 지시서**	7단계
Task list	**공유 할 일 보드**	8단계
Pending / In progress / Completed	**대기 / 진행 중 / 완료**	8단계

원래 용어	쉬운 말	자세한 설명
Dependency	**선행 작업** ("이거 끝나야 시작 가능")	8단계
Self-claim	**스스로 일 가져가기**	8단계
Mailbox / SendMessage	**사내 메신저**	9단계
Idle	**대기 중** (할 일 끝남)	9단계
Plan mode / Plan approval	**설계 먼저 / 설계 결재**	10단계
Permission prompt	**허락 요청**	10단계
Subagent definition	**역할 카드**	11단계
Hook (TeammateIdle, TaskCompleted)	**퇴근·완료 검문소**	12단계
Worktree	**개인 작업 책상 복사본**	13단계
Shutdown	**퇴근시키기**	14단계
Token	**연료비**	14단계

한눈에 보기

혼자 · 심부름꾼 · 팀

클로드에게 큰일을 맡기는 방법은 세 계단입니다. **아래 계단부터** 써보고, 부족할 때만 올라갑니다.

```
3층  🏢 팀 에이전트 (Agent Teams)   팀원끼리 대화·토론·분업          연료비 ★★★
2층  🏃 서브에이전트 (Subagents)     심부름 보내고 결과만 받음        연료비 ★★
1층  🧑 혼자 (한 세션)              한 명이 처음부터 끝까지          연료비 ★
```

상황	추천
순서대로 해야 하는 일, 작은 수정	🧑 혼자
로그·검색 결과가 많아 대화창이 지저분해지는 곁가지 일	🏃 서브에이전트
결과만 필요하고 서로 상의할 필요 없음	🏃 서브에이전트
여러 관점으로 동시에 조사·리뷰	🏢 팀
서로 반박하며 원인을 좁혀야 하는 디버깅	🏢 팀
프론트·백엔드·테스트처럼 파일이 겹치지 않게 나눌 수 있는 개발	🏢 팀

이 자료의 흐름

심부름꾼 복습 → 팀이 필요한 때 → 팀 켜기 → 첫 팀 → 화면 모드 → 팀 구성(이름·모델·인원)
→ 업무 지시서 → 할 일 보드 → 메신저와 토론 → 설계 먼저·권한 → 역할 카드
→ 검문소(hook) → 파일 충돌 피하기 → 비용·운영·한계

1단계

심부름꾼 복습 (Subagents)

📖 이게 뭐야?

Subagent (서브에이전트) = 심부름꾼 메인 클로드가 **곁가지 일**을 다른 클로드에게 보내고, **결과 요약만** 돌려받는 방식입니다. 심부름꾼은 자기 머릿속(컨텍스트)에서 일하기 때문에, 긴 검색 결과나 로그가 메인 대화를 어지럽히지 않습니다.

기본으로 들어 있는 심부름꾼

이름	하는 일	파일 수정
Explore	코드베이스 빠르게 찾아보기	❌ 읽기만
Plan	계획 모드에서 설계 전 조사	❌ 읽기만
general-purpose	조사 + 수정까지 하는 만능	✅

내가 만드는 심부름꾼 = 역할 카드 파일

위치	범위
`.claude/agents/이름.md`	**이 프로젝트**에서만 (git 에 올려 팀과 공유)
`~/.claude/agents/이름.md`	**내 모든 프로젝트**에서

```
---
name: code-reviewer
description: 최근 변경 코드를 품질·보안 관점에서 리뷰할 때 사용
tools: Read, Glob, Grep
model: sonnet
---

너는 코드 리뷰어야. 변경된 코드를 읽고 품질·보안·가독성 문제를
심각도(높음/중간/낮음)와 함께 구체적으로 지적해. 코드는 고치지 마.
```

- `name`, `description` 만 필수. `description` 을 보고 클로드가 **언제 이 심부름꾼을 부를지** 판단합니다
- `tools` 를 적으면 그 도구만 사용 (위 예시는 읽기 전용)
- `model` : `sonnet` / `opus` / `haiku` / `inherit` (메인과 같게)

🛠️ 직접 해보기

1. 클로드에게 부탁해서 만들기 (또는 위 파일을 직접 저장)
2. 부르는 법 3가지:

```
code-reviewer 서브에이전트로 최근 변경 사항을 리뷰해줘
```

```
@"code-reviewer (agent)" src/auth.js 를 봐줘
```

(`@` 를 치면 파일처럼 에이전트 목록이 뜸 → 고르면 반드시 그 심부름꾼이 실행됨)

3. 오래 걸리는 심부름은 **Ctrl + B** 로 **백그라운드**로 보내고 나는 다른 일 하기

❓ 왜 필요해?

팀 에이전트의 팀원도 결국 **이 역할 카드를 그대로 재사용**할 수 있습니다 (11단계). 그리고 팀을 꾸리기 전에 "심부름꾼으로 충분하지 않나?" 를 먼저 따져봐야 합니다.

⚠️ 자주 하는 실수

- `description` 을 대충 씀 → 클로드가 언제 써야 할지 몰라서 안 부름
- 리뷰어에게 `Edit`, `Write` 도구까지 줌 → 리뷰하다 코드를 고쳐버림
- `/agents` 명령으로 만들려고 함 → 최근 버전은 안내문만 보여줌. **클로드에게 부탁하거나 파일을 직접** 만들기

💬 클로드에게 이렇게 말하기

```
.claude/agents/test-runner.md 서브에이전트를 만들어줘.
description: 테스트를 실행하고 실패 원인을 요약할 때 사용.
tools 는 Read, Grep, Glob, Bash 만. 테스트가 실패해도 코드는 고치지 말고 원인과 위치만 보고하게 해줘.
```

2단계

언제 팀이 필요해? (Subagents vs Agent Teams)

📖 이게 뭐야?

	🏃 서브에이전트	🏢 팀 에이전트
머릿속	따로. 결과만 메인에게 돌려줌	따로. **완전히 독립**
대화	메인에게 보고만	**팀원끼리 직접** 메시지
조율	메인이 모든 일을 관리	메시지 + **공유 할 일 보드**로 스스로 조율
잘 맞는 일	결과만 중요한 집중 작업	**토론·협업**이 필요한 복잡한 일
연료비	낮음 (요약만 돌아옴)	**높음** (팀원마다 클로드 한 명)

> 심부름꾼 = **"갔다 와서 보고해"** / 팀 = **"너희끼리 상의해서 해결해"**

팀이 가장 빛나는 4가지 일 (공식 문서 기준)

일	예
조사·리뷰	보안·성능·테스트 리뷰어 3명이 같은 PR 을 각자 다른 눈으로
새 모듈·기능	팀원마다 **자기 파일 묶음**을 맡아 동시에 개발
경쟁 가설 디버깅	원인 후보 5개를 5명이 각자 증명·반박
여러 층에 걸친 변경	프론트엔드·백엔드·테스트를 각자 맡음

팀을 쓰면 오히려 손해인 일

- 1 → 2 → 3 **순서대로** 해야 하는 일
- **같은 파일**을 여럿이 고쳐야 하는 일
- 서로 **기다려야 하는 의존성**이 많은 일

❓ 왜 필요해?

팀은 연료비가 비싼 대신 **"토론"** 이 됩니다. 토론이 필요 없는 일에 팀을 쓰면 비용만 몇 배가 됩니다. 이 판단이 팀 에이전트 사용법의 절반입니다.

⚠️ 자주 하는 실수

- "빨리 끝내고 싶어서" 순서가 있는 일을 팀에게 → 팀원들이 서로 기다리느라 더 느림
- 작은 버그 하나에 5명 팀 → 회의비만 나감

💬 클로드에게 이렇게 말하기

```
이 작업을 혼자 / 서브에이전트 / 팀 에이전트 중 어떤 방식으로 하는 게 좋을지 판단해줘.
판단 기준(순서 의존성, 같은 파일 수정 여부, 관점이 여러 개 필요한지, 비용)을 표로 보여줘. 아직 실행하지 마.
```

3단계

팀 기능 켜기

📖 이게 뭐야?

팀 에이전트는 **실험 기능이라 기본으로 꺼져** 있습니다. 환경 변수 `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS` 를 `1` 로 하면 켜집니다. 꺼져 있으면 클로드는 팀원을 뽑지도, 제안하지도 않습니다.

settings.json 이란? Claude Code 설정 파일입니다.

위치	범위
`~/.claude/settings.json`	내 컴퓨터의 **모든** 프로젝트
`.claude/settings.json`	**이 프로젝트**만 (git 으로 공유)
`.claude/settings.local.json`	이 프로젝트, **나만** (git 에 안 올림)

🛠️ 직접 해보기

1. Claude Code 최신 버전으로

```
claude update     # Claude Code 를 최신 버전으로 업데이트 (팀 기능은 실험 기능이라 최신 버전 권장)
claude --version  # 설치된 Claude Code 버전 확인
```

2. 설정 파일에 추가 (프로젝트에서만 켜려면 `.claude/settings.json`)

```
{
  "env": {
    "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1"
  }
}
```

3. `claude` 를 실행 (일반 대화 모드)

끄고 싶을 때: 값을 `"0"` 으로 바꾸고 저장 → 다음에 뽑는 에이전트부터 다시 일반 심부름꾼으로

❓ 왜 필요해?

켜두면 클로드가 **스스로 판단해서 팀을 꾸리기도** 합니다. 이름 붙인 심부름꾼이 자동으로 팀원으로 바뀌기 때문입니다. 그래서 **팀 실습하는 프로젝트에서만** 켜는 게 안전합니다 (전체 설정 `~/.claude/settings.json` 보다 프로젝트 설정 추천).

⚠️ 자주 하는 실수

- `claude -p "..."` (한 번 실행하고 끝나는 모드)에서 팀을 부름 → **대화형 세션에서만** 팀원이 생김
- JSON 문법 오류 (쉼표, 따옴표) → 설정이 통째로 무시됨
- 전체 설정에 켜놓고 잊어버림 → 평소 작업에서도 팀이 생겨 토큰이 새어 나감

💬 클로드에게 이렇게 말하기

```
이 프로젝트의 .claude/settings.json 에 env 로 CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1 을 추가해줘.
기존 설정이 있으면 지우지 말고 합쳐줘. 저장 후 파일 전체를 보여줘.
```

4단계

첫 팀 만들기 (Spawn · 팀원 현황판)

📖 이게 뭐야?

Spawn = 팀원 뽑기 특별한 명령어는 없습니다. **"팀원 N명을 뽑아서 이렇게 나눠 해줘"** 라고 말로 하면 팀장이 팀원을 만들고, 할 일 보드를 채우고, 일을 나눠주고, 마지막에 결과를 합칩니다.

Agent panel = 팀원 현황판 입력창 **아래**에 팀원 목록이 생깁니다.

키	하는 일
↑ / ↓	팀원 고르기
Enter	그 팀원의 대화 기록 보기 + **그 팀원에게 직접 말하기**
Esc	선택 해제. 팀원 화면을 보는 중이면 그 팀원의 현재 작업을 **멈춤**
x	고른 팀원 **중지**
Ctrl + T	**할 일 보드** 보이기/숨기기

- 할 일이 끝나 **대기 중(idle)** 인 팀원 줄은 모두가 쉬면 30초 뒤 숨겨짐 → 없어진 게 아니라 **숨은 것**. 이름으로 메시지를 보내면 다시 나타남
- 대기 중인 팀원이 3명을 넘으면 `2 idle agents` 처럼 한 줄로 묶임 → Enter 로 펼치기

🛠️ 직접 해보기

공식 문서의 첫 예시입니다. 세 역할이 **서로 기다릴 필요 없이** 동시에 생각할 수 있어서 연습에 좋습니다.

```
코드베이스 전체의 TODO 주석을 추적하는 CLI 도구를 설계하려고 해.
팀원 3명을 뽑아서 서로 다른 각도로 검토해줘:
한 명은 사용자 경험(UX), 한 명은 기술 구조, 한 명은 반대 의견을 내는 악마의 변호인.
```

→ 현황판에서 ↑↓ 로 팀원을 골라 Enter → 각자 무슨 생각을 하는지 구경 → Esc 로 돌아오기

❓ 왜 필요해?

팀장만 보고 있으면 팀원이 **엉뚱한 방향으로 오래 달리는 걸** 놓칩니다. 현황판으로 중간중간 들여다보고 방향을 잡아주는 게 팀 운영의 기본입니다.

⚠️ 자주 하는 실수

- 클로드가 팀 대신 **일반 심부름꾼**을 씀 → 현황판에 같이 보여서 헷갈림. "**agent team 으로**, 팀원을 뽑아서" 라고 다시 분명히 요청
- 팀원 줄이 사라져서 "죽었다" 고 생각 → 30초 뒤 **숨겨진 것**
- 너무 단순한 일을 줌 → 클로드가 팀이 필요 없다고 판단하고 혼자 함

💬 클로드에게 이렇게 말하기

```
이번엔 서브에이전트 말고 agent team 으로 해줘. 팀원 3명을 뽑고 이름은 ux, arch, critic 으로 해줘.
```

5단계

화면 모드 (한 화면 · 분할 화면)

📖 이게 뭐야?

모드	쉬운 말	특징	필요한 것
In-process (기본값)	한 화면	모든 팀원이 내 터미널 하나 안에. 현황판에서 골라서 봄	없음. **어떤 터미널이든**
Split panes	분할 화면	팀원마다 칸이 따로 → 모두의 진행을 **동시에** 봄	**tmux** 또는 **iTerm2**(+ `it2` CLI)

설정 값 (`teammateMode`)

값	뜻
`"in-process"`	항상 한 화면 (기본값)
`"auto"`	tmux 안이거나 iTerm2 면 분할, 아니면 한 화면
`"tmux"`	분할 화면 (tmux/iTerm2 자동 감지)
`"iterm2"`	iTerm2 분할 화면 직접 사용

🛠️ 직접 해보기

`~/.claude/settings.json`:

```
{
  "teammateMode": "auto"
}
```

이번 세션만 바꾸기:

```
claude --teammate-mode auto  # 이번 세션만 화면 모드 지정 (auto = tmux/iTerm2 면 분할 화면, 아니면 한 화면)
```

❓ 왜 필요해?

팀원이 3명 이상이면 한 화면에서 번갈아 보기가 번거롭습니다. mac 이고 tmux 를 쓸 줄 알면 분할 화면이 편합니다. **처음엔 기본값(한 화면)으로 충분합니다.**

⚠️ 자주 하는 실수

- **Windows Terminal, VS Code 내장 터미널, Ghostty** 에서 분할 화면 시도 → **지원 안 됨**. 한 화면 모드를 쓰세요
- 분할 화면을 켰는데 안 나뉨 → `which tmux` 로 tmux 가 설치되어 있는지 확인
- 세션을 끝냈는데 tmux 세션이 남음 → `tmux ls` 로 찾아서 `tmux kill-session -t 이름`

💬 클로드에게 이렇게 말하기

```
내 터미널 환경에서 agent teams 분할 화면(split panes)을 쓸 수 있는지 확인해줘.
안 되면 이유와 한 화면 모드에서 팀원을 보는 키 조작을 정리해줘.
```

6단계

팀 구성 정하기 (인원 · 이름 · 모델)

📖 이게 뭐야?

인원: 제한은 없지만 공식 권장은 **3~5명**.

- 팀원 수만큼 연료비가 **정비례**로 늘어남
- 많을수록 **회의(조율)** 가 늘고 충돌 가능성도 커짐
- 일이 15개여도 팀원 **3명**이면 충분. 팀원 1명당 **할 일 5~6개**가 적당
- "집중한 3명이 흩어진 5명보다 낫다"

이름: 팀장이 이름을 붙이고, 팀원끼리 **이름으로** 메시지를 주고받습니다. 내가 이름을 정해주면 나중에 "**reviewer 에게** 이거 물어봐" 처럼 부르기 쉽습니다.

모델: 팀원 모델은 이 순서로 정해집니다.

1. 내 요청에 적은 모델 ("팀원은 Sonnet 으로")
2. 역할 카드(11단계)의 `model`
3. `CLAUDE_CODE_SUBAGENT_MODEL` 환경 변수
4. 팀장과 같은 모델

> 💡 비용을 아끼려면 **팀원은 Sonnet** (공식 비용 가이드 권장). 팀장은 판단이 중요하니 그대로.

🛠️ 직접 해보기

```
팀원 4명을 뽑아서 이 4개 모듈을 동시에 리팩터링해줘. 팀원은 모두 Sonnet 으로.
이름은 auth, books, server, tests 로 하고, 각자 자기 이름의 모듈만 맡게 해.
```

❓ 왜 필요해?

인원·모델을 안 정하면 클로드가 알아서 정하는데, 비싼 모델로 많이 뽑을 수 있습니다. **작게 시작해서 필요할 때만 늘리기.**

⚠️ 자주 하는 실수

- "많을수록 빠르겠지" → 5명 넘으면 조율 비용이 이득보다 커짐
- 이름을 안 정함 → 팀원 이름이 랜덤이라 나중에 지목하기 어려움
- 팀원 화면에서 `/model` 로 모델을 바꾸려 함 → 팀원 모델은 **뽑을 때 고정**. 그 명령은 팀장 모델을 바꿈

💬 클로드에게 이렇게 말하기

```
이 작업에 팀원이 몇 명 필요한지 먼저 제안해줘. 팀원별 역할, 맡을 파일, 예상 할 일 개수를 표로.
3명 이하로 가능하면 3명 이하로. 내가 OK 하면 그때 뽑아줘.
```

7단계

업무 지시서 잘 쓰기 (Spawn prompt)

📖 이게 뭐야?

팀원은 팀장과 나눈 대화를 모릅니다. 이게 제일 중요합니다.

팀원이 자동으로 아는 것	팀원이 모르는 것
CLAUDE.md, MCP 서버, 스킬 (프로젝트 기본 정보)	**팀장과 내가 지금까지 나눈 대화**
팀장이 뽑을 때 준 업무 지시서	내가 앞에서 설명한 배경·결정 사항

그래서 팀원을 뽑을 때 주는 **업무 지시서(spawn prompt)** 에 필요한 걸 다 적어야 합니다.

좋은 업무 지시서 5칸

```
① 역할:    너는 ∞ 담당이다
② 범위:    src/auth/ 폴더만 본다 (다른 파일은 건드리지 않는다)
③ 배경:    이 앱은 JWT 토큰을 httpOnly 쿠키에 저장한다
④ 할 일:   토큰 처리, 세션 관리, 입력 검증을 점검한다
⑤ 결과물:  문제마다 심각도(높음/중간/낮음) + 파일:줄 + 고치는 방법을 보고한다
```

🛠️ 직접 해보기

```
보안 리뷰어 팀원을 뽑고 이 지시서를 줘:
"src/auth.js 와 server.js 의 인증 부분을 보안 관점에서 리뷰해.
비밀번호 저장 방식, 토큰 생성·검증, 입력 검증을 중점으로.
문제마다 심각도(높음/중간/낮음), 파일:줄, 재현 방법, 고치는 방법을 표로 보고해. 코드는 고치지 마."
```

❓ 왜 필요해?

지시서가 짧으면 팀원은 **처음부터 코드베이스를 뒤지며** 배경을 추측합니다. 연료비도 들고 방향도 틀리기 쉽습니다. 반대로 너무 길면 그것도 전부 팀원 머릿속에 들어가니, **5칸만 정확히**.

⚠️ 자주 하는 실수

- "아까 말한 대로 해" → 팀원은 "아까"를 모름
- 범위를 안 정함 → 두 팀원이 **같은 파일**을 고쳐서 서로 덮어씀 (13단계)
- 결과물 형식을 안 정함 → 팀장이 합칠 때 제각각이라 정리가 안 됨

💬 클로드에게 이렇게 말하기

```
팀원을 뽑기 전에, 각 팀원에게 줄 업무 지시서를 역할/범위/배경/할 일/결과물 5칸으로 먼저 보여줘.
내가 확인하고 OK 하면 그때 뽑아줘.
```

8단계

공유 할 일 보드 (Task list)

📖 이게 뭐야?

Task list = 팀 전체가 보는 할 일 보드 (칸반 보드 같은 것)



개념	쉬운 말
3가지 상태	대기 → 진행 중 → 완료
Dependency (선행 작업)	"API 구현이 끝나야 화면 연결 시작 가능". 선행 작업이 끝나면 **자동으로 잠금 해제**
팀장이 배정	"이 일은 books 에게" 라고 내가 팀장에게 말하면 그대로
스스로 가져가기 (self-claim)	팀원이 일을 끝내면 **배정 안 된, 잠금 안 된** 다음 일을 스스로 가져감
동시에 같은 일 가져가기 방지	파일 잠금으로 두 명이 같은 일을 잡는 걸 막음

Ctrl + T 로 보드를 열고 닫습니다.

🛠️ 직접 해보기

```
일을 시작하기 전에 할 일 보드부터 만들어줘. 팀원 1명당 할 일 5~6개가 되게 잘게 나누고,
선행 작업 관계(무엇이 끝나야 무엇을 시작할 수 있는지)도 설정해줘. 보드를 보여주고 내 OK 를 기다려.
```

→ **Ctrl + T** 로 보드를 열어두고 진행 상황 구경

❓ 왜 필요해?

보드가 없으면 팀장이 일일이 말로 나눠줘야 하고, 누가 뭘 하는지 **나도 팀장도 헷갈립니다.** 보드는 팀의 "지금 상태"를 한눈에 보여주는 유일한 화면입니다.

⚠️ 자주 하는 실수

- 할 일을 너무 크게 → 팀원이 **오래 혼자 달리다** 엉뚱한 결과. 함수 하나, 테스트 파일 하나, 리뷰 하나 크기로
- 할 일을 너무 작게 → 일보다 **조율 비용**이 더 큼
- 보드가 멈춘 것 같음 → 팀원이 완료 표시를 **깜빡**하는 경우가 있음(알려진 한계). 실제로 끝났는지 확인하고 팀장에게 "○○ 팀원에게 완료 표시하라고 해줘"

💬 클로드에게 이렇게 말하기

```
할 일 보드에서 3번 작업이 계속 진행 중인데 실제로 끝났는지 확인해줘.
끝났으면 완료로 바꾸고, 그 작업을 기다리던 일들이 풀렸는지도 알려줘.
```

9단계

팀원끼리 대화하기 (메신저 · 토론)

📖 이게 뭐야?

Mailbox = 사내 메신저

- 팀원이 보낸 메시지는 **자동으로 배달**됩니다. 팀장이 확인하러 다닐 필요 없음
- 팀원이 일을 마치고 **대기 상태**가 되면 → 팀장에게 **자동 알림 + 최종 결과**가 갑니다
- 메시지는 **이름으로 한 명에게**. 전체 공지는 한 명씩 보냄

나도 팀원에게 직접 말할 수 있다 현황판에서 팀원을 고르고 **Enter** → 그 팀원에게 바로 입력. 팀장을 거치지 않고 추가 지시·질문·방향 수정이 가능합니다.

> 팀원 화면에서 `/compact`, `/clear`, `/rewind` 는 **팀장 대화에** 적용되므로 확인을 한 번 더 묻습니다.

🔥 **경쟁 가설 토론 (팀의 필살기)** 클로드 한 명은 **처음 찾은 그럴듯한 원인에 닻을 내립니다(anchoring).** 그래서 일부러 **서로 반박하게** 만듭니다.

```
사용자들이 "메시지 하나 보내면 앱이 꺼진다"고 해.
팀원 5명을 뽑아서 각자 다른 가설을 조사하게 해줘.
과학 토론처럼 서로의 가설을 반박하려고 노력하게 하고,
합의된 결론을 findings.md 에 정리해줘.
```

→ 반박에서 **살아남은 가설**이 진짜 원인일 가능성이 높습니다.

🔒 **안전장치** 팀원끼리 보낸 메시지는 **"다른 클로드가 보낸 것"** 으로 표시됩니다. 팀원은 **나 대신 허락(권한)을 줄 수 없고**, 거절당한 작업을 다른 팀원에게 떠넘겨 우회할 수도 없습니다.

❓ 왜 필요해?

서브에이전트는 각자 결과만 보고하고 끝나지만, 팀은 **서로의 결과를 보고 반박·보완**합니다. 이게 팀을 쓰는 가장 큰 이유입니다.

⚠️ 자주 하는 실수

- 토론을 시키면서 "결론 정리" 를 안 시킴 → 메시지만 오가고 끝남. **결과 파일(findings.md)** 을 정해주기
- 팀원에게 직접 말할 수 있다는 걸 모름 → 모든 걸 팀장을 거쳐 전달 (느리고 왜곡됨)

💬 클로드에게 이렇게 말하기

```
critic 팀원에게 전달해줘: "arch 의 구조안에서 가장 약한 부분 2개를 찾아 반박하고, arch 와 직접 대화해서 합의안을 만들어."
합의되면 docs/decision.md 에 정리하게 해.
```

10단계

설계 먼저, 권한은 내가 (Plan mode · Permissions)

📖 이게 뭐야?

설계 먼저 (Plan mode) 팀장을 **계획 모드**로 둔 상태에서 팀원을 뽑으면, 그 팀원은 **읽기만 하며 설계부터** 합니다. 설계가 끝나면 팀장에게 **설계 결재 요청**을 보내고, 결재되면 구현을 시작합니다.

> ⚠️ **주의**: 공식 문서 기준, 이 설계 결재는 팀장 세션이 **나에게 묻지 않고 바로 승인**합니다. 설계를 **내 눈으로** 보고 싶으면 "설계를 파일로 써서 나에게 먼저 보여주고, 내가 OK 할 때까지 구현하지 마" 라고 지시서에 적으세요.

계획 모드 켜기: **Shift + Tab** 을 눌러 모드를 돌리다 `plan mode` 가 보이면 멈춤

권한 (Permissions)

- 팀원은 **팀장의 권한 모드**를 그대로 물려받습니다
- 팀원의 **허락 요청은 팀장 화면에 뜹니다** → 거기서 내가 승인
- 뽑을 때 팀원마다 다른 권한을 줄 수는 없음 (뽑은 뒤 개별 변경은 가능)
- 팀장을 `--dangerously-skip-permissions` 로 켜면 **모든 팀원도** 허락 없이 실행 → 위험

🛠️ 직접 해보기

1. **Shift + Tab** 으로 팀장을 계획 모드로
2. 이렇게 요청:

```
architect 팀원을 뽑아서 인증 모듈 리팩터링 설계를 하게 해줘.
설계는 docs/auth-plan.md 에 쓰고, 내가 OK 하기 전에는 구현하지 마.
```

3. 허락 요청이 너무 많으면 → 팀원 뽑기 **전에** 자주 쓰는 명령(`npm test` 등)을 미리 허용

❓ 왜 필요해?

팀원 여럿이 동시에 코드를 고치기 시작하면 되돌리기가 어렵습니다. **설계 → 확인 → 구현** 순서를 지키면 엉뚱한 방향으로 다 같이 달리는 사고를 막습니다.

⚠️ 자주 하는 실수

- 설계 결재를 내가 하는 줄 앎 → 팀장이 **자동 승인**함. 내가 보고 싶으면 지시서에 명시
- 허락 요청 폭탄 → 미리 허용 목록(permissions)에 테스트·린트 명령 넣기
- 귀찮아서 모든 허락을 끔 → 팀원 여러 명이 확인 없이 명령 실행

💬 클로드에게 이렇게 말하기

```
.claude/settings.json 의 permissions.allow 에 "Bash(npm test:*)", "Bash(node --test:*)" 를 추가해줘.
팀원들이 테스트를 돌릴 때마다 허락을 묻지 않게 하려는 거야. 기존 설정은 유지해줘.
```

11단계

역할 카드 재사용 (Subagent definitions → 팀원)

📖 이게 뭐야?

1단계에서 만든 **심부름꾼 역할 카드**(`.claude/agents/이름.md`)를 **팀원으로도** 쓸 수 있습니다. 보안 리뷰어, 테스트 담당 같은 역할을 **한 번 정의해서** 심부름꾼으로도, 팀원으로도 재사용합니다.

```
security-reviewer 에이전트 타입으로 팀원을 뽑아서 인증 모듈을 점검하게 해줘.
```

역할 카드에서 팀원에게 적용되는 것 / 안 되는 것 (한 화면 모드 기준)

카드 항목	팀원에게
`tools`	✅ 그 도구만 사용 (+ 메신저·할 일 보드 도구는 자동 추가)
`model`	✅ 요청에 모델을 안 적으면 카드의 모델 사용
`disallowedTools`	✅ 그 도구 제외
본문(지시 내용)	✅ 기본 지시에 **덧붙여짐**
`skills`	❌ 적용 안 됨 (프로젝트·내 설정의 스킬을 씀)
`mcpServers`	❌ 한 화면 모드에선 무시 (분할 화면에선 적용)

🛠️ 직접 해보기

`.claude/agents/security-reviewer.md`

```
---
name: security-reviewer
description: 인증·입력 검증·비밀 정보 노출 등 보안 문제를 리뷰할 때 사용
tools: Read, Glob, Grep
model: sonnet
color: red
---

너는 보안 리뷰어야. 코드를 고치지 말고, 문제마다
심각도(높음/중간/낮음), 파일:줄, 공격 시나리오, 고치는 방법을 표로 보고해.
추측이면 "추측"이라고 표시해.
```

❓ 왜 필요해?

매번 업무 지시서에 "너는 보안 리뷰어고, 고치지 말고, 표로..." 를 쓰지 않아도 됩니다. **역할 카드 = 팀의 직무 기술서.** git 에 올리면 다른 사람도 같은 팀원을 씁니다.

⚠️ 자주 하는 실수

- 카드에 `skills` 를 적고 팀원이 쓸 거라 기대 → 팀원에겐 적용 안 됨
- 리뷰 카드에 `Edit` 도구를 줌 → 리뷰어가 코드를 직접 고침
- `.claude/teams/teams.json` 같은 파일로 팀을 미리 정의하려 함 → **그런 설정 파일은 없음**. 역할은 `.claude/agents/` 로

💬 클로드에게 이렇게 말하기

```
.claude/agents/ 에 팀원용 역할 카드 3개를 만들어줘: security-reviewer, perf-reviewer, test-reviewer.
모두 읽기 전용(Read, Glob, Grep), model 은 sonnet, 결과는 심각도/파일:줄/고치는 방법 표로 보고하게.
```

12단계

퇴근·완료 검문소 (Hooks)

📖 이게 뭐야?

Hook = 정해진 순간에 자동으로 실행되는 검문소 (하네스 강의의 그 hook 입니다) 팀 에이전트에는 전용 검문소가 3개 있습니다. 스크립트가 **종료 코드 2** 로 끝나면 → 그 행동을 막고, 에러 메시지(stderr)를 클로드에게 피드백으로 줍니다.

Hook	언제	exit 2 면
TeammateIdle	팀원이 **일 끝내고 쉬려고** 할 때	못 쉬고 **계속 일함**
TaskCreated	할 일이 **만들어질** 때	그 할 일 **생성 취소**
TaskCompleted	할 일을 **완료로 표시**하려 할 때	완료 **거부**

🛠️ 직접 해보기 — "테스트 통과 못 하면 완료 금지"

`.claude/settings.json`

```
{
  "env": { "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1" },
  "hooks": {
    "TaskCompleted": [
      {
        "hooks": [
          { "type": "command", "command": "npm test 1>&2 ¦¦ exit 2" }
        ]
      }
    ]
  }
}
```

- `npm test 1>&2` = 테스트 결과를 stderr 로 → 실패하면 그 내용이 팀원에게 피드백으로 감
- `¦¦ exit 2` = 테스트가 실패하면 exit 2 → **완료 표시 거부** → 팀원이 다시 고침

❓ 왜 필요해?

팀원은 "다 했어요!" 라고 하고 쉬려는 경향이 있습니다. 사람이 일일이 확인하는 대신 **기계가 자동으로 검사**하게 하면, 팀원 여럿이 동시에 일해도 품질이 지켜집니다.

⚠️ 자주 하는 실수

- 검사 스크립트가 exit **1** 로 끝남 → 막지 못함. 막으려면 **2**
- 오래 걸리는 검사(전체 E2E)를 TaskCompleted 에 → 완료할 때마다 몇 분씩 대기. 빠른 단위 테스트만
- 테스트가 원래 깨져 있는 상태에서 hook 을 켬 → **아무 일도 완료 못 함**. 먼저 테스트를 초록으로

💬 클로드에게 이렇게 말하기

```
.claude/settings.json 에 TaskCompleted hook 을 추가해줘. npm test 를 돌려서 실패하면 exit 2 로 완료를 막고,
실패 내용은 stderr 로 보내서 팀원이 보고 고칠 수 있게. 기존 env 설정은 유지해줘.
```

13단계

파일 충돌 피하기 (파일 나누기 · Worktree)

📖 이게 뭐야?

팀 에이전트는 팀원마다 작업 폴더를 따로 주지 않습니다. 모두 **같은 폴더**에서 일합니다. 그래서 두 팀원이 같은 파일을 고치면 **서로 덮어씁니다.**

해결 1 — 파일 주인 정하기 (팀 에이전트의 정답)

```
팀원 books   →  src/books.js, test/books.test.js
팀원 auth    →  src/auth.js,  test/auth.test.js
팀원 web     →  public/index.html
팀장          →  server.js (여러 팀원이 손대는 파일은 팀장이 마지막에 합침)
```

해결 2 — Worktree (개인 작업 책상 복사본) git worktree = 같은 저장소의 **작업 폴더를 하나 더** 만드는 것. 파일·브랜치가 따로라서 절대 안 겹칩니다. (Git 강의의 브랜치를 **폴더째** 나눈 것)

- 내가 직접 클로드 세션을 여러 개 띄울 때:

```
claude --worktree feature-auth  # 작업 폴더 복사본(worktree)을 만들고 그 안에서 클로드 시작 → .claude/worktrees/feature-auth/ 에 새 브랜치 worktree-feature-auth (.claude/worktrees/feature-auth/ 에서 새 브랜치 worktree-feature-auth 로 시작)
```

- **심부름꾼**은 역할 카드에 `isolation: worktree` 를 넣으면 자기만의 임시 worktree 에서 일함
- ⚠️ **팀원은 worktree 로 자동 분리되지 않습니다** → 해결 1(파일 나누기)로

상황	방법
팀 에이전트로 개발	**팀원마다 파일 주인** 정하기
심부름꾼 여러 명이 수정	역할 카드에 `isolation: worktree`
내가 세션 여러 개를 직접 운영	`claude --worktree 이름`

🛠️ 직접 해보기

```
팀원 3명으로 개발할 건데, 시작 전에 파일 소유표를 만들어줘.
팀원마다 수정 가능한 파일 목록을 정하고, 겹치는 파일(server.js 등)은 팀장만 고치게 해.
이 규칙을 각 팀원의 업무 지시서에 넣어줘.
```

💡 `claude --worktree` 를 쓴다면 `.gitignore` 에 `.claude/worktrees/` 를 추가하고, `.env` 같은 git 에 없는 파일은 `.worktreeinclude` 파일에 적어두면 새 worktree 에 자동 복사됩니다.

❓ 왜 필요해?

"두 팀원이 같은 파일을 동시에 고치면 덮어쓴다" 는 공식 문서가 직접 경고하는 문제입니다. 팀 개발에서 가장 흔하고, 가장 찾기 어려운 사고입니다.

⚠️ 자주 하는 실수

- 파일 주인 없이 "알아서 나눠서 해" → 공통 파일(라우터, 설정)에서 충돌
- 팀원도 worktree 로 분리될 거라 기대 → 안 됨
- 작업 전에 **커밋을 안 함** → 덮어쓰기 사고가 나면 돌아갈 곳이 없음. 팀 시작 전 **git commit 필수**

💬 클로드에게 이렇게 말하기

```
팀 작업을 시작하기 전에 지금 상태를 git commit 해줘. 그리고 각 팀원이 끝날 때마다
자기 파일만 git add 해서 "팀원이름: 한 일" 형식으로 커밋하게 해줘. 다른 팀원 파일은 add 하지 마.
```

14단계

비용 · 운영 · 한계

📖 이게 뭐야?

🔥 연료비 줄이기 (공식 비용 가이드)

방법	이유
팀원은 Sonnet	능력 대비 비용 균형
팀은 작게 (3~5명)	연료비 ≈ 팀원 수에 비례
업무 지시서는 핵심만	지시서 전부가 팀원 머릿속에 처음부터 들어감
끝난 팀원은 퇴근시키기	살아 있는 팀원은 계속 연료를 씀
처음엔 조사·리뷰부터	코드를 안 쓰는 일이 조율 문제가 적음

`/usage` 로 지금까지 쓴 양을 확인할 수 있습니다.

퇴근시키기 (Shutdown)

```
researcher 팀원에게 퇴근하라고 해줘
```

→ 팀장이 퇴근 요청 → 팀원이 승인하면 종료 (하던 작업을 마무리한 뒤라 **조금 걸림**). 세션이 끝나면 팀 폴더는 자동 정리.

팀장이 직접 일을 해버릴 때

```
팀원들이 맡은 일을 끝낼 때까지 기다려. 직접 구현하지 마.
```

팀장이 너무 일찍 끝냈다고 할 때 → "할 일 보드에 남은 일이 있어. 계속 진행해"

⚠️ 알려진 한계 (실험 기능)

한계	대처
`/resume`, `/rewind` 로 팀원은 복원 안 됨	다시 열면 "새 팀원을 뽑아줘"
할 일 상태가 늦게 바뀜	실제 완료 확인 후 수동으로 완료 처리
퇴근이 느림	하던 일을 끝내고 퇴근함. 기다리기
세션당 팀 1개	팀을 바꾸려면 새 세션
팀원이 팀원을 못 뽑음	팀 관리는 팀장만
팀장 교체 불가	처음 세션이 끝까지 팀장
팀원은 뽑을 때 권한 모드 고정	뽑은 뒤 개별 변경
분할 화면은 tmux/iTerm2 만	Windows·VS Code 는 한 화면 모드

🛠️ 문제 해결표

증상	해결
팀원이 안 보임	현황판(입력창 아래)에서 ↑↓. 숨겨진 거면 이름으로 메시지
팀 대신 심부름꾼이 생김	"agent team 으로 팀원을 뽑아줘" 라고 다시
원하지 않는데 팀이 생김	설정에서 `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS` 를 `"0"`
허락 요청 폭탄	자주 쓰는 명령을 permissions.allow 에 미리 등록
팀원이 에러 후 멈춤	팀원 화면에서 직접 지시하거나, 대체 팀원 뽑기
같은 파일 덮어쓰기	파일 주인 정하기 + 작업 전 커밋
분할 화면이 안 됨	`which tmux`, Windows/VS Code 터미널은 미지원

💬 클로드에게 이렇게 말하기

```
지금 팀원 중 할 일이 끝난 팀원은 모두 퇴근시키고, 남은 팀원과 할 일 보드 상태를 표로 정리해줘.
```

부록 A

상황별 프롬프트 모음

상황	프롬프트
팀 만들기	`팀원 3명을 뽑아서 ∞를 서로 다른 각도로 검토해줘: A 관점, B 관점, C 관점`
팀 명시	`서브에이전트 말고 agent team 으로 해줘`
이름·모델	`이름은 ux, arch, critic 으로, 팀원은 모두 Sonnet 으로`
지시서 먼저	`뽑기 전에 팀원별 업무 지시서(역할/범위/배경/할 일/결과물)를 보여줘`
할 일 보드	`팀원 1명당 할 일 5~6개로 나누고 선행 작업도 설정해줘`
역할 카드	`security-reviewer 에이전트 타입으로 팀원을 뽑아줘`
설계 먼저	`설계를 docs/plan.md 에 쓰고 내가 OK 하기 전엔 구현하지 마`
토론	`서로의 가설을 반박하게 하고 합의된 결론을 findings.md 에 정리해줘`
기다려	`팀원들이 끝낼 때까지 기다려. 직접 구현하지 마`
파일 주인	`팀원마다 수정 가능한 파일을 정하고 겹치는 파일은 팀장만 고쳐`
퇴근	`∞ 팀원에게 퇴근하라고 해줘`

상황	프롬프트
정리	`팀원들 결과를 하나의 보고서로 합쳐서 report.md 에 써줘`

부록 B

설정 · 키 치트시트

하고 싶은 것	방법
팀 기능 켜기	settings.json → `"env": { "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1" }`
팀 기능 끄기	같은 값을 `"0"`
화면 모드	settings.json → `"teammateMode": "in-process"` / `"auto"` / `"tmux"` / `"iterm2"`
이번 세션만 화면 모드	`claude --teammate-mode auto`
팀원 고르기 / 보기	현황판에서 ↑ ↓ → **Enter**
팀원 작업 멈추기	팀원 화면에서 **Esc**
팀원 중지	고른 상태에서 **x**
할 일 보드	**Ctrl + T**
계획 모드	**Shift + Tab** 으로 순환
심부름꾼 백그라운드로	**Ctrl + B**
심부름꾼 지목	`@` → 에이전트 선택

하고 싶은 것	방법
역할 카드 위치	`.claude/agents/이름.md` (프로젝트) / `~/.claude/agents/이름.md` (전체)
worktree 세션	`claude --worktree 이름`
사용량 확인	`/usage`
남은 tmux 세션 정리	`tmux ls` → `tmux kill-session -t 이름`

부록 C

팀 운영 10계명

1. 팀을 꾸리기 전에 **"혼자나 심부름꾼으로 안 되나?"** 먼저 묻는다
2. 처음엔 **조사·리뷰**처럼 코드를 안 쓰는 일로 시작한다
3. 팀원은 **3~5명**, 모델은 **Sonnet**
4. 팀원은 대화를 모른다 → **업무 지시서 5칸** (역할·범위·배경·할 일·결과물)
5. 할 일은 **팀원 1명당 5~6개**, 결과물이 분명한 크기로
6. **파일 주인**을 정하고, 공통 파일은 팀장만 고친다
7. 팀 시작 전 **git commit**, 끝나면 팀원별로 커밋
8. 토론을 시키면 **결론 파일**을 정해준다
9. **검문소(hook)** 로 테스트 통과 전엔 완료를 막는다
10. 중간중간 **현황판을 들여다보고**, 끝난 팀원은 **퇴근시킨다**